Predicting Real Estate Prices on a U.S. Listings Dataset Using XGBoost and Random Forest

Francesco Vanella
Politecnico di Torino
s349141
s349141@studenti.polito.it

***Abstract*—This report aims to analyze the effectiveness of machine learning techniques in predicting real estate listing prices. In particular, it proposes a method to identify and extract the most relevant factors influencing prices, and then compares the performance of two regression models, namely Extreme Gradient Boosting (XGBoost) and Random Forest, based on their MAE scores.**

I. Problem Overview

The task of the competition is to build a regression model capable of accurately estimating real estate listing prices, based on a variety of property-related features.

The dataset provided comprises two subsets:

- A development set containing 79,589 samples with the target variable, which we will use to train our models.
- An evaluation set of 19,898 samples, for which the target is not provided and on which we will test the trained models.

We begin with a preliminary analysis to understand the structure of the dataset.

Several features contain a significant number of missing values, as shown in Table I. Missing values associated with *amenities* and *pets_allowed* may reflect properties without relevant amenities or where pets are not allowed. For other features, the missing entries could correspond to unreported values. Therefore, it will be necessary to adopt the appropriate strategies to handle them during the preprocessing step.

By inspecting the distributions of the variables, we identify many numerical variables, in particular *square footage* and the target variable *price*, exhibiting highly skewed distributions (see Figure 1), which often are associated with the presence of outliers. This is one of the reasons why we will use Random Forest and XGBoost, as both models are known to be robust to noise and outliers. Among the categorical attributes, *currency*, *fee*, *pets_allowed* and *price_type* have distributions with "almost non-zero variance", and as a result they could be removed prior to model trainings.

Shifting our attention to the geographical distribution of the listings, although the development dataset contains tens of thousands of them, they are not uniformly distributed across the United States. They are instead concentrated in approximately 6,941 unique geographic locations. The reason could be that many of them belong to the same buildings or skyscrapers. Moreover, within each state, and within each city, complex pricing patterns emerge, with certain areas showing significantly higher prices than others. These factors could decrease the informative power of *cityname* and *state*, and instead could increase that of latitude and *longitude*.

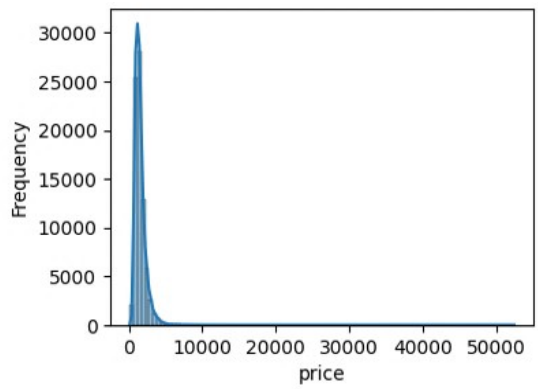

Fig. 1. Histogram of *price*

TABLE I
Missing values percentages

Feature	Missing values (%)
amenities	16.1%
bathrooms	0.064%
bedrooms	0.112%
pets_allowed	60.8%
address	92.0%
cityname	0.294%
state	0.294%
latitude	0.029%
longitude	0.029%

Another important aspect of the dataset is the listing source: most of the listings are made by one source (RentDigs.com), while only a small fraction originates from other websites. In addition, they often use repetitive templates to write titles and bodies, and typically post listings in bulk uploads, eliminating any meaningful temporal pattern in *date*. This reduces the information that could be extracted from these features.

Like the listing source, the textual features *title* and *body* contain as a whole very little information, since the majority are made through templates. However, it makes sense to focus only on the most important keywords in terms of correlation with respect to the price. Figure 2 shows the keywords of the *title* feature that have the highest correlation in absolute value with the price. We note that we do not consider keywords that

contain city names or other similar geographical information because this information is already contained in other features such as *cityname*.

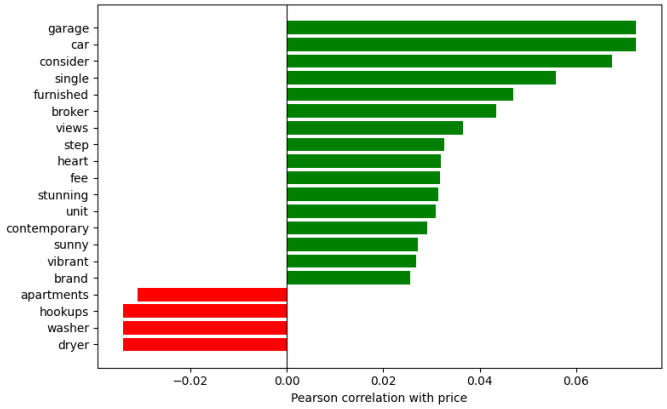

Fig. 2. Correlation of the top 20 title keywords with price

Once the nature of the variables has been analyzed, it is time to investigate the Pearson correlation among the numerical variables. Figure 3 shows a heatmap of the correlations.
We can immediately notice that *id* has the lowest correlation with the target variable *price*, instead *square_feet* has the highest. In addition, we see that there are groups of correlated variables (e.g. *square_feet* and *bathrooms*). We will use these correlations to select the preprocessing strategies in Section II.

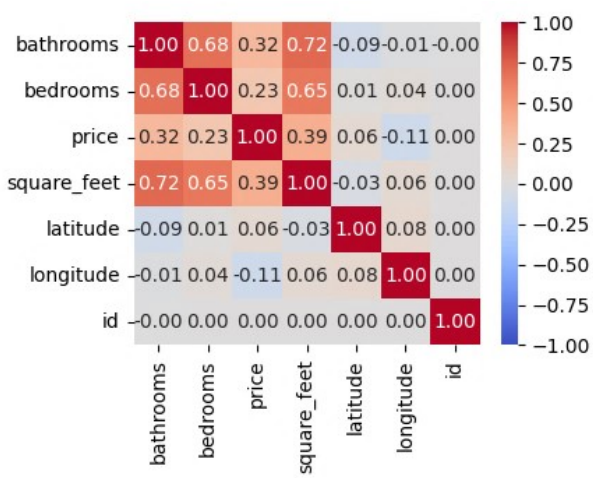

Fig. 3. Heatmap of correlations

II. Proposed approach

A. Preprocessing

Based on the observations made in the previous section, we remove the following variables: *address*, *currency*, *fee*, *price type*, *pets allowed*, *date*, and *id*.
We also remove *body* and *title* as whole texts, but we keep the top 25 keywords that have the highest absolute correlation with the price for both features and that do not contain the names of cities or states, since they are already covered by the *cityname* and *state* features. The number of keywords to keep is set heuristically based on the analysis of the number of keywords in general and on the correlation values, but obviously can be tuned to find the optimal value. We encode their presence in a listing with one-hot encoding.
We then address the issue of missing values. For the *amenities* feature, we simply encoded the absence of information with the string 'Missing'. Instead, based on the correlations found before, we impute missing values in bathrooms and bedrooms using a K-Nearest Neighbors (KNN) approach, averaging the values of the three nearest listings in terms of *square footage*. For geographical variables, we can adopt the same approach as before to infer missing values in *cityname* and *state*, looking at the three nearest neighbors in terms of *latitude* and *longitude*. In contrast, there are few instances with missing values in both *cityname* and *state* and *latitude* and *longitude*, which could not be reliably inferred. In these cases, we infer missing values with default values.
To further decrease the dimensionality of the datasets, we choose to aggregate, for each state, cities with fewer than 100 listings, with a default name, such as "Others_CA" in the case of cities in California, and to discard *state*, since most of its information is already captured by *cityname*, which provides a more fine-grained level of detail.
To address the skewed distribution of the target variable *price*, we apply a logarithmic transformation. This transformation helps to reduce skewness and stabilize variance, which could improve the model performance.
Taking advantage of the observations made in the previous chapter, we can also add a new feature that reports the number of listings in the same geographical location, as a sort of measure of density.
Finally, we applied one-hot encoding to all remaining categorical variables.

B. Model selection

The following algorithms were selected:

- *Random Forests*: is an ensemble model, which makes predictions based on the output of multiple independent decision trees. It decorrelates base trees by training them on random subsets of input variables at each node of the tree, as well as on randomly chosen subset of data. It does not require normalization of input data, but it necessitates imputation of missing values.
  The use of this type of model is supported by related works [1].
- *XGBoost*: is a gradient boosting decision trees algorithm. It works by adding sequentially new trees to correct errors made by existing trees. When it adds new trees, it tries to minimize the loss using a gradient descent algorithm. It does not require normalization of input data and automatically learns how to best handle missing data. This model has shown to work well in the real-estate domain [1] [2].

C. Hyperparameters tuning

For tuning the models, we decide to keep 80% of the development set for training, and the remaining part for validation.

The most challenging aspect when working with XGBoost is parameter tuning, as there is no predefined subset of values to search over. For this reason, we have found the recommendations available in [3] particularly helpful.

To optimize the baseline hyperparameters, we can perform a grid search considering different combinations of parameters. The hyperparameter sets chosen are summarized in Table II and Table III.

Once the hyperparameter sets are selected, we can assess their performance on the validation set.

TABLE II
HYPERPARAMETERS CONSIDERED FOR RANDOM FOREST

Parameter	Values
n_estimators	100, 300, 500
min_samples_leaf	1, 10

TABLE III
HYPERPARAMETERS CONSIDERED FOR XGBOOST

Parameter	Values
n_estimators	8000, 15000
learning_rate	0.02
max_depth	11, 13
subsample	0.8
colsample_bytree	0.6, 0.8
min_child_weight	0.01
reg_alpha	0, 0.1

III. RESULTS

The best configuration of hyperparameters for the Random Forest was found to be {n_estimators=500, min_samples_leaf=1}, with MAE $\approx$ 166.3. While the best configuration for XGBoost was {n_estimators=15,000, learning_rate=0.02 max_depth=11, subsample=0.8, colsample_bytree=0.6, min_child_weight=0.01, reg_alpha=0}, with MAE $\approx$ 142.6. This error represents the 9.3% of the mean price.

As a side note, we show the most important features for the XGBoost decision making. The Figure 4 shows the top 10 features in terms of gain importance extracted from the best XGBoost model.

It is evident that the geographical features are the most important for XGBoost. This is understandable: states like California or cities like New York have usually significantly higher prices with respect to other less economical or important U.S. states/cities.

We then tested the optimal sets of hyperparameters found on the evaluation set. Random Forest obtained a public leaderboard score of 152.791, while XGBoost reached a public leaderboard score of 136.239.

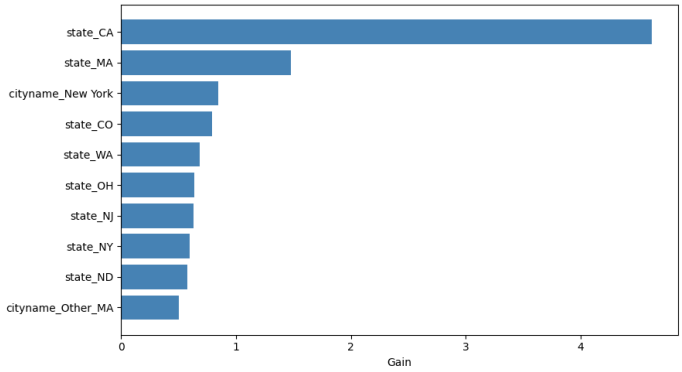

Fig. 4. Top 10 features in terms of gain importance from the best XGBoost model

IV. DISCUSSION

The results showed that XGBoost consistently outperformed Random Forest in both validation and evaluation datasets. This highlights XGBoost's ability to better capture complex and nonlinear relationships in tabular data, thanks to its gradient boosting framework.

Hyperparameter tuning is more challenging for XGBoost than for Random Forest, as the former is more sensitive to changes in hyperparameters, but it allows a better optimization. Instead, Random Forest shows more susceptibility to redundant or weakly informative features, and therefore could require more intensive preprocessing to achieve the performance of XGBoost on complex tabular data.

Another important point we would like to highlight is that XGBoost proved to be significantly faster than Random Forest. Even with thousands of estimators, its training time is comparable with that of Random Forest, which was trained with far fewer estimators.

To further improve the results, several other techniques could be considered, such as conducting a more extensive hyperparameter search and making better use of the geographical information, for example, through the application of geographical clustering techniques.

REFERENCES

[1] R. Ritu, "Machine learning techniques for house price prediction: A literature review," 12 2023.

[2] H. Sharma, H. Harsora, and B. Ogunleye, "An optimal house price prediction algorithm: Xgboost," *Analytics*, vol. 3, no. 1, pp. 30–45, 2024.

[3] J. Brownlee, *XGBoost With Python: Gradient Boosted Trees with XGBoost and scikit-learn*. Machine Learning Mastery, 2016.